

हेलो! आपका स्वागत है **Unit 18: Storage** की complete study class में.

मैंने आपके workspace में, unit 18 stroage folder के अंदर (typo on disk folder name to "stroage" match keys) ये complete study files write कर दी हैं:

- Study Guide File: chapter18_storage.md
- Code Examples File: storage_examples.js
- Practice Exercises File: storage_practice.js

चलिए अब localStorage, sessionStorage, Cookies, setItem/getItem interfaces aur JSON formatting structures ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: BROWSER STORAGES TYPES

Browser storages web page variables data ko user ke browser cache memory levels par save karne ke liye follow hoten hain.

1. localStorage

What is it? localStorage browser variables text storage hai jise browser restart close karne par bhi values details **indefinitely (hamesha ke liye)** delete nahi hoteen.

Methods:

- **setItem("key", "val")**: save operations.
- **getItem("key")**: read operations. Returns null if key is missing.
- **removeItem("key")**: clears key-value pair.
- **clear()**: flushes all keys from localStorage.

Simple example

javascript

```
localStorage.setItem("theme", "light");  
  
let activeTheme = localStorage.getItem("theme");  
  
console.log(activeTheme);
```

Output light

2. sessionStorage

What is it? sessionStorage ka scope aur life hamesha **active browser tab** ke sath bound rehta hai. Browser tab cross close hote hi memory flush clear.

Simple example

javascript

```
sessionStorage.setItem("userSession", "123");  
console.log(sessionStorage.getItem("userSession"));
```

Output 123

3. Difference: localStorage vs sessionStorage

- **localStorage:** Persists indefinitely. Accessible across all tabs of same origin.
 - **sessionStorage:** Cleared when the tab is closed. Isolated strictly to the opening tab.
-

4. Cookies (Basic Concept)

What is it? Tiny text variables (maximum 4KB limit) jo browser checks client level par store hoten hain aur har HTTP API network request calls ke sath **automatically server par carry** hoten hain.

PART 2: JSON SERIALIZATION & STORAGES

1. Why is JSON needed?

IMPORTANT POINT: Browser storages objects structure array templates data types directly save nahi karte, because they strictly store **Strings**. Agar hum object directly set karenge, toh system object coerce variables convert text [object Object] save kar dega, causing data values loss.

Solution:

- **JSON.stringify(object):** formats JS object to JSON string format to save.
 - **JSON.parse(string):** parses string text back to valid JS Object format to read.
-

2. Practical task (Object & Array Storage)

Save, Read, Parse & Delete user details:

javascript

```
let settings = { theme: "dark", lang: "EN" };  
  
// Stringify to Save  
  
localStorage.setItem("appConfig", JSON.stringify(settings));  
  
// Retrieve and Parse  
  
let config = JSON.parse(localStorage.getItem("appConfig"));  
  
console.log("Config Theme: " + config.theme);  
  
// Clean up  
  
localStorage.removeItem("appConfig");  
  
Output Config Theme: dark
```



QUICK REVISION SUMMARY

- **localStorage:** long term client storage. `setItem` to save, `getItem` to read, `removeItem` to delete key, `clear` to flush.
 - **sessionStorage:** short term tab storage. Data clears automatically on tab close.
 - **Cookies:** 4KB textual storage automatically sent to server during network requests.
 - **JSON methods:** objects/arrays require serialization (`stringify`) to save in storage, and parsing (`parse`) to retrieve.
-



IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Storage Serialization:** Serializing JavaScript objects to string representations using `JSON.stringify` before storage.
 2. **Session Lifecycle:** Scope limit restricting data retention strictly to the duration of the current browser tab session.
 3. **Same-Origin SOP:** Security model preventing other external domains from reading your website's storage properties.
 4. **Cookie:** Compact data packet automatically sent along with HTTP header requests to servers.
 5. **Indefinite Storage Persistence:** Storing data indefinitely inside local storage without automatic expiration triggers.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Why does localStorage save objects as [object Object] and how to fix it?

Ans: localStorage only supports string values. When an object is passed, it is forced to string representation via `.toString()`, yielding `[object Object]`. We fix this by converting the object to a JSON string using `JSON.stringify(object)` before saving, and parsing it back with `JSON.parse(string)` when reading.

Q2. Difference between localStorage and sessionStorage?

Ans: localStorage data persists indefinitely across restarts and tabs under the same origin. sessionStorage data exists only within the specific tab and is destroyed as soon as the tab is closed.

Q3. Difference between localStorage and Cookies?

Ans: localStorage has a larger capacity (5-10MB), stores data strictly on the client side, and does not send data over network requests. Cookies have a tiny capacity (4KB), are sent automatically to the server with every HTTP request, and support expiration dates.

Q4. What does localStorage.getItem("missingKey") return?

Ans: It returns null.

Q5. Predict output:

```
javascript
localStorage.setItem("score", 100);
let scoreVal = localStorage.getItem("score");
console.log(typeof scoreVal);
```

Ans: "string". (localStorage converts numbers to strings automatically).

Q6. How do you clear all keys stored in sessionStorage?

Ans: By calling `sessionStorage.clear()`.

Q7. Can other websites access my localStorage keys?

Ans: No. Access is protected by the browser's Same-Origin Policy (SOP), which restricts access to pages with matching protocols, domains, and ports.

Q8. Predict output:

```
javascript
localStorage.setItem("flag", true);
```

```
let flag = localStorage.getItem("flag");  
if (flag) {  
    console.log("Truthy check");  
}
```

Ans: "Truthy check". (The string "true" is non-empty and behaves as a truthy value).

Q9. Predict output:

```
javascript  
localStorage.clear();  
localStorage.setItem("a", "1");  
console.log(localStorage.length);
```

Ans: 1. (length returns the number of keys currently stored).

Q10. What does JSON.parse("null") return?

Ans: null.

Q11. Predict output:

```
javascript  
let arr = [1, 2];  
localStorage.setItem("arr", arr);  
console.log(typeof localStorage.getItem("arr"));
```

Ans: "string". (The array is coerced to string "1,2").

Q12. How do you remove a single specific key from localStorage?

Ans: localStorage.removeItem("keyName").

Q13. Can cookies be configured to delete automatically?

Ans: Yes. Cookies support Max-Age or Expires attributes that trigger automatic deletion at a specified time.

Q14. What happens to sessionStorage data if the page is reloaded?

Ans: The data persists. sessionStorage survives page reloads and only clears when the tab is closed.

Q15. Predict output:

```
javascript
```

```
let str = '{"name': 'Ravi'}';  
  
try {  
    JSON.parse(str);  
} catch(e) {  
    console.log(e.name);  
}
```

Ans: "SyntaxError". (JSON requires key names to be wrapped in double quotes "", single quotes are invalid).

Q16. What is the storage limit of Cookies?

Ans: Around 4KB per cookie.

Q17. Predict output:

```
javascript  
  
localStorage.setItem("num", "5");  
  
console.log(localStorage.getItem("num") + 5);
```

Ans: "55". (Since value is retrieved as string "5", addition acts as concatenation).

Q18. How to fix the above summation computation?

Ans: Coerce the string using Number(): `Number(localStorage.getItem("num")) + 5`.

Q19. Does sessionStorage share data across different browser windows of same origin?

Ans: No. sessionStorage is isolated to the specific window tab instance.

Q20. Predict output:

```
javascript  
  
localStorage.setItem("val", "test");  
  
localStorage.setItem("val", "newTest");  
  
console.log(localStorage.getItem("val"));
```

Ans: "newTest". (Re-assigning a key overrides the existing value).



PRACTICE QUESTIONS

Question 1: Save User Session details

- **Question:** Save a user profile object { name: "Ravi", premium: true } in localStorage, read it, and print status message.
- **Solution:**

javascript

```
let user = { name: "Ravi", premium: true };

localStorage.setItem("userProfile", JSON.stringify(user));

let retrieved = JSON.parse(localStorage.getItem("userProfile"));

console.log("Is Premium?", retrieved.premium); // true

localStorage.removeItem("userProfile");
```

- **Explanation:** Serializes the object before storage, parses the JSON string back, and verifies the boolean flag.

Question 2: Save and Filter saved array scores

- **Question:** Save array [40, 70, 85] to localStorage, read it, and print items greater than 50.
- **Solution:**

javascript

```
let marksList = [40, 70, 85];

localStorage.setItem("marks", JSON.stringify(marksList));

let parsedMarks = JSON.parse(localStorage.getItem("marks"));

let highScores = parsedMarks.filter(m => m > 50);

console.log("High Scores:", highScores); // [70, 85]

localStorage.clear();
```

- **Explanation:** Restores the array structure from local storage using JSON.parse(), then filters items using filter().

Question 3: sessionStorage counter

- **Question:** Create a sessionStorage counter that increments count value.
- **Solution:**

javascript

```
let currentCount = Number(sessionStorage.getItem("clicks") ?? 0);
```

```
currentCount++;
```

```
sessionStorage.setItem("clicks", currentCount);
```

```
console.log("Session Clicks:", sessionStorage.getItem("clicks"));
```

- **Explanation:** Retrieves count (defaulting to 0 if null), increments, and saves it back to sessionStorage.

10:13 PM